Modul 3 Konsep Dasar Pemrograman Berorientasi Objek

Tujuan Praktikum
1. Praktikan dapat mengerti konsep dasar Object Oriented Programming. 2. Praktikan dapat membandingkan antara pemrograman prosedural dengan pemrograman berorientasi Object. 3. Praktikan dapat memahami Kelas, Object, Method dan Constructor, serta mengimplementasikannya dalam suatu program Java.

3.1 Pendahuluan

Sebelum membahas tentang kelas terlebih dahulu Kita perlu mengetahui konsep pemrograman berorientasi objek. Suatu sistem yang dibangun dengan metode berorientasi objek adalah sebuah sistem yang komponennya di-enkapsulasi menjadi kelompok data dan fungsi, yang dapat mewarisi atribut dan sifat dari komponen lainnya dan komponen-komponen tersebut saling berinteraksi satu sama lain.

Beberapa karakteristik utama dalam pemrograman berorientasi objek yaitu:

3.1.1 Abstraksi

Abstraksi pada dasarnya adalah **menemukan hal-hal yang esensial pada suatu objek** dan mengabaikan hal-hal yang sifatnya insidental. Kita menentukan apa ciri-ciri (atribut) yang dimiliki oleh objek tersebut serta apa saja yang bisa dilakukan (metode/perilaku) oleh objek tersebut.

Contohnya adalah abstraksi pada manusia.

Ciri-ciri (atribut) : punya tangan, berat, tinggi, dll. (kata benda, atau kata sifat).

Fungsi (perilaku) : makan, minum, berjalan, dll. (kata kerja).

3.1.2 Enkapsulasi

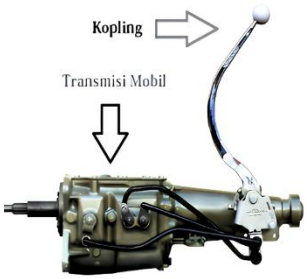
Pengkapsulan adalah **proses pemaketan data objek bersama *method-method*nya**. Manfaat utama pengkapsulan adalah penyembunyian rincian-rincian implementasi dari pemakai/objek lain yang tidak berhak. Enkapsulasi memprotek suatu proses dari kemungkinan interferensi atau penyalahgunaan dari luar sistem.

Fungsi dari enkapsulasi adalah:

a. Penyembunyian data
   Penyembunyian data (*data hiding*) mengacu perlindungan data internal objek. Objek tersebut disusun dari antarmuka *public Method* dan *private* data. Manfaat utama adalah bagian internal dapat berubah tanpa mempengaruhi bagian-bagian program yang lain.

b. Modularitas
   Modularitas (*modularity*) berarti objek dapat dikelola secara **independen**. Karena kode sumber bagian internal objek dikelola secara terpisah dari antarmuka, maka Kita bebas melakukan modifikasi yang tidak menyebabkan masalah pada bagian-bagian lain. Manfaat ini mempermudah mendistribusikan objek-objek di sistem.

Contoh ilustrasi pada mobil :



Mobil memiliki sistem transmisi dan sistem ini menyembunyikan proses yang terjadi di dalamnya tentang bagaimana cara mobil tersebut bekerja, mulai dari bagaimana cara mobil mengatur percepatan dan apa yang dilakukan terhadap mesin untuk mendapat percepatan tersebut.

Kita sebagai pengguna hanya cukup memindah-mindahkan tongkat transmisi (kopling) untuk mendapatkan percepatan yang diinginkan. Tongkat transmisi adalah satu-satunya *interface* yang digunakan dalam mengatur sistem transmisi mobil tersebut. Kita tidak dapat menggunakan pedal rem untuk mengakses sistem transmisi tersebut dan sebaliknya dengan mengubah-ubah sistem transmisi Kita tidak akan bisa menghidupkan radio mobil atau membuka pintu mobil. Konsep yang sama dapat pula Kita terapkan dalam pemrograman orientasi objek.

3.1.3 Pewarisan (Inheritance)

Pewarisan adalah **proses penciptaan kelas baru** (yang disebut subclass atau kelas turunan) **dengan mewarisi karakteristik** dari kelas yang telah ada (*superclass* atau kelas induk), ditambah karakteristik unik kelas baru itu. Karakteristik unik tersebut bisa merupakan perluasan atau spesialisasi dari *superclass*. Kelas turunan akan mewarisi anggota-anggota suatu kelas yang berupa data (atribut) dan fungsi (operasi) dan pada kelas turunan memungkinkan menambahkan data serta fungsi yang baru. Secara praktis berarti bahwa jika *superclass* telah mendefinisikan perilaku yang Kita perlukan, maka Kita tidak perlu mendefinisi ulang perilaku itu, Kita cukup membuat kelas yang merupakan *subclass* dari *superclass* yang dimaksud.

3.1.4 Polymorphisms

Polymorphism berasal dari bahasa Yunani yang berarti **banyak bentuk**. Konsep ini memungkinkan digunakannya **antarmuka (*interface*) yang sama untuk memerintah suatu objek agar dapat melakukan aksi** atau tindakan yang mungkin secara prinsip sama tapi secara proses berbeda. Seringkali *Polymorphism* disebut dengan "satu interface banyak aksi". Mekanisme *Polymorphism* dapat dilakukan dengan dengan beberapa cara, seperti *Overloading Method*, *Overloading Constructor*, maupun *Overriding Method*. Semua akan dibahas pada bab selanjutnya.






Contoh ilustrasi pada mobil. Mobil yang ada di pasaran terdiri atas berbagai tipe dan merek, tetapi semuanya memilki *interface* kemudi yang hampir sama seperti setir, tongkat transmisi, pedal gas, dll. Jika Kita dapat mengemudikan satu jenis mobil saja dari satu merek tertentu, maka boleh dikatakan Kita dapat mengemudikan hampir semua jenis mobil yang ada karena semua mobil tersebut menggunakan *interface* yang sama. Misal, ketika Kita menekan pedal gas pada mobil A, Kita telah berhasil menggerakkan mobil A tersebut dengan percepatan yang tinggi. Sebaliknya, ketika Kita menggunakan mobil B dan sama-sama menekan pedal gas, mobil B ini bergerak agak lambat. Jadi, dapat disimpulkan dengan **sama-sama** Kita **menekan pedal gas** pada dua mobil ini akan didapatkan **hasil** yang **berbeda** pada keduanya.

3.1.5 Message (Komunikasi Antar Objek)

Objek yang bertindak sendirian jarang berguna, kebanyakan objek memerlukan objek-objek lain untuk melakukan banyak hal. Oleh karena itu, objek-objek tersebut saling berkomunikasi dan berinteraksi lewat message. Ketika berkomunikasi, suatu objek mengirim pesan (dapat berupa pemanggilan *method*) untuk memberitahu agar objek lain melakukan sesuatu yang diharapkan. Seringkali pengiriman *message* juga disertai informasi untuk memperjelas apa yang dikehendaki. Informasi yang dilewatkan beserta *message* adalah parameter message

Fakultas Informatika
School of Computing
Telkom University
informatics lab

3.2 Kelas

Kelas adalah cetak biru (rancangan) dari objek. Ini berarti Kita bisa membuat banyak objek dari satu macam kelas. Kelas mendefiniskan sebuah tipe dari objek. Di dalam kelas Kita dapat mendeklarasikan variabel dan menciptakan *Object* (instansiasi). Sebuah kelas mempunyai anggota(member) yang terdiri atas atribut dan *method*. Atribut adalah semua *field* identitas yang Kita berikan pada suatu kelas, misal kelas manusia memiliki *field* atribut berupa nama, maupun umur. *Method* dapat Kita artikan sebagai semua fungsi ataupun prosedur yang merupakan perilaku (*behaviour*) dari suatu kelas. Dikatakan fungsi bila *method* tersebut melakukan suatu proses dan mengembalikan suatu nilai (*return value*), dan dikatakan prosedur bila *method* tersebut hanya melakukan suatu proses dan tidak mengembalikan nilai (*void*).

Contoh pendeklarasian kelas:

```
class <classname>
{
      //declaration of data member
      //declaration of Methods
}
```

Kata class merupakan *keyword* pada Java yang digunakan untuk mendeklarasikan kelas dan **<classname>** digunakan untuk memberikan nama kelas. Sebagai contoh, dibawah ini terdapat pendeklarasian kelas Mahasiswa yang mempunyai *field* nama, nim dan *method* **getNamaMahasiswa()**.

```
class Mahasiswa
{
      String nama;                    //data anggota
      int nim;                        //data anggota
      void getNamaMahasiswa() {};     //Method
}
```

Setelah mengetahui apa itu kelas , selanjutnya Kita perlu mengetahui *Object* dan *Java Modifier.*

3.3 Object

Object (objek) secara lugas dapat **diartkan sebagai insatansiasi atau hasil ciptaan dari suatu kelas** yang telah dibuat sebelumnya. Dalam pengembangan program orientasi objek lebih lanjut, sebuah objek dapat dimungkinkan terdiri atas objek-objek lain. Seperti halnya objek mobil terdiri atas mesin, ban, kerangka mobil, pintu, karoseri dan lain-lain. Atau, bisa jadi sebuah objek merupakan turunan dari objek lain sehingga mewarisi sifat-sifat induknya. Misal motor dan mobil merupakan kendaraan bermotor, sehingga motor dan mobil mempunyai sifat-sifat yang dimiliki oleh kelas kendaraan bermotor dengan spesifikasi sifat-sifat tambahan sendiri.

Contoh *Object* yang lain: Komputer, TV, mahasiswa, ponsel, lbuku, dan lainnya.

Contoh pembuatan Objek dalam java:

```
Manusia objMns1 = new Manusia(); //membuat objek manusia
```

Fakultas Informatika
School of Computing

3.4 Field

Field adalah sebuah atribut. *Field* bisa berupa sebuah variabel kelas, variabel objek, variabel *Method* objek atau parameter dari sebuah fungsi. *Field* merupakan anggota dari kelas yang digunakan untuk menyimpan data. Terdapat dua *field* dalam kelas , dibawah ini adalah contoh kelas yang terdapat 2 jenis *field* tersebut.

```
public class Circle {
  public static final double PI= 3.14159;
  public static double radiansToDegrees(double rads) {
    return rads * 180 / PI;
  }
  public double r;
  public double area() {
    return PI * r * r;
  }
  public double circumference() {
    return 2 * PI * r;
  }
}
```

a. Class Field

Field yang dikaitkan dengan kelas dimana dia didefinisikan, bukan dengan sebuah *instance* dari kelas. Di bawah ini menyatakan pendeklarasian kelas *field* pada kelas **Circle** adalah:

```
public static final double PI= 3.14159;
```

Static *modifier* diatas menyatakan bahwa *field* tersebut merupakan *class field*. *Field* ini berkaitan dengan kelas itu sendiri tidak dengan *instance* dari kelas. Berdasarkan kelas **Circle** maka jika terdapat *Method* dari kelas **Circle** maka mengakses variabel **PI** dengan **Circle.PI**

b. Instance Field

Field apapun yang dideklarasikan tanpa menggunakan *static modifier* . Contoh *instance field* yang terdapat pada kelas **Circle** adalah:

```
public double r;
```

Field ini merupakan *field* yang terkait dengan *instance* kelas bukan dengan kelas itu sendiri. Berdasarkan kelas **Circle** , setiap objek **Circle** yang dibuat memiliki salinan sendiri *field* r. Contohnya r menyatakan jari-jari lingkaran. Jadi setiap objek **Circle** dapat memiliki jari-jari sendiri dari semua objek lingkaran lainnya.

School of Computing
Telkom University
informatics lab

3.5 Method

Method dikenal juga sebagai suatu *function* dan *procedure*. Dalam OOP, *Method* digunakan untuk memodularisasi program melalui pemisahan tugas dalam suatu kelas. Pemanggilan *Method* menspesifikasikan nama *Method* dan menyediakan informasi (parameter) yang diperlukan untuk melaksanakan tugasnya.

Deklarasi *Method* untuk yang mengembalikan nilai (fungsi)

```
[modifier] Type-data namaMethod(parameter1,parameter2,…parameterN)
{
      Deklarasi-deklarasi dan proses ;
      return nilai-kembalian
}
```

Type-data merupakan tipe data dari nilai yang dapat dikembalikan oleh *Method* ini.

Deklarasi *Method* untuk yang tidak mengembalikan nilai (prosedur).

```
[modifier] void namaMethod(parameter1,parameter2,…parameterN)
{
      Deklarasi-deklarasi dan proses ;
}
```

Contoh implementasi *Method*:

```
public int jumlahAngka(int x, int y)
{
       int z=x+y;
       return z;
}
```

Ada dua cara melewatkan argumen ke *Method*, yaitu:

a. Melewatkan secara Nilai (*Pass by Value*)

   Digunakan untuk argumen yang mempunyai tipe data primitif (Byte, short, int, long, float, double, char, dan boolean). Prosesnya adalah *compiler* hanya menyalin isi memori (pengalokasian suatu variabel), dan kemudian menyampaikan salinan tersebut kepada *Method*. Isi *memory* ini merupakan data "sesungguhnya" yang akan dioperasikan.

   Karena hanya berupa salinan isi *memory*, maka perubahan yang terjadi pada variabel akibat proses di dalam *method* tidak akan berpengaruh pada nilai variabel asalnya.

b. Melewatkan secara Referensi (Pass by Reference)

   Digunakan pada *array* dan objek. Prosesnya isi memory pada variabel *array* dan objek merupakan penunjuk ke alamat memory yang mengandung data sesungguhnya yang akan dioperasikan. Dengan kata lain, variabel *array* atau objek menyimpan alamat *memory* bukan isi *memory*. Akibatnya, setiap perubahan variabel di dalam *method* akan mempengaruhi nilai pada variabel asalnya.

Fakultas Informatika
Telkom University
informatics lab

Contoh program:

```
//file TestPass.java
class TestPass {
      int i,j;

      TestPass(int a,int b) {
       i = a;
       j = b;
    }
      //passed by value dengan parameter berupa tipe data primitif
      void calculate(int m,int n) {
            m = m*10;
            n = n/2;
      }
      //passed by reference dengan berupa tipe data class
      void calculate(TestPass e) {
            e.i =  e.i*10;
            e.j =  e.j/2;
      }
}
```

```
// file PassedByValue.java
class PassedByValue {
     public static void main(String[] args) {
  int x,y;
        TestPass z;
        z = new TestPass(50,100);
        x = 10;
        y = 20;

        System.out.println("Nilai sebelum passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);

        //passed by value
        z.calculate(x,y);
        System.out.println("Nilai sesudah passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);
        System.out.println("Nilai sebelum passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);

        //passed by reference
        z.calculate(z);

        System.out.println("Nilai sesudah passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);
   }
}
```

Telkom University
informatics lab

Output dari program diatas

```
Nilai sebelum passed by value :
x = 10
y = 20

Nilai sesudah passed by value :
x = 10
y = 20

Nilai sebelum passed by reference :
z.i = 50
z.j = 100

Nilai sesudah passed by reference :
z.i = 500
z.j = 50
```

Keterangan:

Pada saat pemanggilan *method* **`calculate()`** dengan metode *pass by value*, hanya nilai dari variabel *x* dan *y* saja yang dilewatkan ke variabel *m* dan *n* , sehingga perubahan pada variabel *m* dan *n* tidak akan mengubah nilai dari variabel *x* dan *y*. Sedangkan pada saat pemanggilan *method* **`calculate()`** dengan metode *pass by reference* yang menerima parameter bertipe kelas Test. Pada waktu Kita memanggil *method* **`calculate()`**, nilai dari variabel *z* yang berupa referensi ke

objek sesungguhnya dilewatkan ke variabel *a* , sehingga variabel *a* menunjukkan ke objek yang sama dengan yang ditunjuk oleh variabel *z* dan setiap perubahan pada objek tersebut dengan menggunakan variabel *a* akan terlihat efeknya pada variabel *z* yang terdapat pada kode yang memangil *method* tersebut.

3.6 Keyword ”this”

Dalam Java terdapat suatu besaran referensi khusus, disebut this, yang digunakan dalam method yang dirujuk untuk objek yang sedang berlaku. Nilai *this* merujuk pada objek di mana method yang sedang berjalan dipanggil.

Contoh implementasi program:

```
class Lagu {
      private String pencipta;
      private String judul;
      public void IsiParam(String judul,String pencipta) {
            this.judul  = judul;
            this.pencipta = pencipta;
      }
      public void cetakKeLayar() {
            if(judul==null&& pencipta==null) return;
            System.out.println("Judul  :  "  +  judul  +",  pencipta  :  "  +
pencipta);
      }
}
class DemoLagu {
public static void main(String[] args) {
       Lagu a = new Lagu();
       a.IsiParam("God Will Make A Way","Don Moen ");
       a.cetakKeLayar();
    }
}
```

Keluaran dari program:

```
Judul : God Will Make A Way, pencipta : Don Moen
```

Keterangan :

Perhatikan pada *method* **`IsiParam()`** di atas. Di sana Kita mendeklarasikan nama variabel yang menjadi parameternya sama dengan nama variabel yang merupakan *property* dari kelas Lagu (judul dan pencipta). Dalam hal ini, Kita perlu menggunakan *keyword* this (*keyword* ini merefer ke objek itu sendiri) agar dapat mengakses *property* judul dan pencipta di dalam *method* **`IsiParam()`** tersebut. Apa yang terjadi jika pada *method* **`IsiParam()`** *keyword* this dihilangkan?

Keyword this juga dapat digunakan untuk memanggil suatu konstruktor dari konstruktor lainnya.

Contoh Program:

```
public class Buku {
    private String pengarang;
    private String judul;
    private Buku() {
        this("Rumah Kita", "GoodBles");
          // this ini digunakan untuk memanggil konstruktor yang
          // menerima dua parameter
    }
    private Buku(String judul,String pengarang)
    {
        this.judul = judul;
        this.pengarang = pengarang;
    }
    private void cetakKeLayar()
    {
        System.out.println("Judul : " + judul + " Pengarang : " +
pengarang);
    }
    public static void main(String[] args)
    {
        Buku a,b ;
        a = new Buku("Jurassic Park", "Michael Chricton");
        b = new Buku();
        a.cetakKeLayar();
        b.cetakKeLayar();
    }
 }
```

Keluaran dari program diatas:

```
Judul : Jurassic Park Pengarang : Michael Chricton
Judul : Rumah Kita Pengarang : GoodBles
```

Pada contoh program di atas, Kita juga dapat mengambil pelajaran bahwa penggunaan *modifier private* pada *constructor* mengakibatkan *constructor* tersebut hanya dapat dikenali dalam satu kelas saja, sehingga pembuatan objek hanya dapat dilakukan dalam lokal kelas tersebut.

Catatan penting lainnya : bahwa *method* **`void main(String[] args)`** haruslah bersifat **public static.**